

TestMender: Evidence-Guided Security Test Agent

Fares Trad
EECS
York University
Toronto, Canada
fares610@my.yorku.ca

Emmanuel Nzeakor
EECS
York University
Toronto, Canada
enzeakor@yorku.ca

Md. Ehsan Shahmi Chowdhury
EECS
York University
Toronto, Canada
ehsansh@yorku.ca

I. INTRODUCTION

Software vulnerabilities continue to threaten critical systems despite advances in static analysis and fuzzing. While tools such as Cppcheck and SonarQube can detect suspicious patterns, and fuzzers like AFL++ can trigger crashes, they do not produce reusable regression tests that capture security flaws in an interpretable form. In parallel, Large Language Models (LLMs) such as GPT-4o and Gemini have shown promise in code reasoning and automated repair, yet their potential for security test generation remains largely unexplored. We identify a gap: current AI research focuses on fixing vulnerabilities (e.g., patch generation), while the generation of targeted vulnerability-revealing tests remains underdeveloped. This motivates TestMender, a framework that investigates whether LLMs can create executable security tests when guided by concrete vulnerability evidence, including taint warnings, crash inputs, and stack traces.

II. BACKGROUND AND RELATED WORKS

A. LLM-Based Code Repair (CodeMender, Code Agents) [3]

CodeMender is an agent-based LLM framework that iteratively proposes code patches and verifies their correctness using automated tools and feedback loops. By integrating environment interaction, validation tools, and patch review, it demonstrates that LLMs can operate as autonomous code repair agents rather than passive text generators. However, its scope is limited to fixing code. It does not produce artifacts for testing or validating vulnerabilities.

B. Static and Dynamic Analysis (SonarQube, AFL++)

Traditional static analysis tools such as Cppcheck and SonarQube can detect tainted data flows, buffer misuse, and other security patterns, but they only produce diagnostic warnings without generating runnable test artifacts. Similarly, dynamic fuzzers like AFL++ discover crashing inputs through mutation-based exploration, yet the resulting crashes remain raw data and are not converted into structured, reproducible test cases. Neither approach provides executable harnesses that explicitly demonstrate and validate vulnerabilities, leaving a gap in evidence-driven security testing.

C. Juliet 1.1 C/C++ and Java Test Suite [2]

The Juliet 1.1 Test Suite is a large, public-domain collection of over 81,000 small, synthetic C/C++ and Java programs, each containing a single, specific, and well-documented security flaw (CWE). Developed by the NSA and distributed by NIST, its primary purpose is to benchmark and assess the effectiveness of static analysis tools and other software assurance products by testing their ability to find known vulnerabilities.

D. Automated Security Test Generation with Formal Threat Models [1]

This paper presents a method for automatically generating security tests from formal threat models. The approach uses a type of high-level Petri net, called Predicate/Transition (PrT) nets, to model threats and systematically generate all possible “attack paths.” The authors validated this approach on two real-world systems: the Magento e-commerce platform and the FileZilla FTP server. Their experiments showed that the automatically generated security tests were highly effective, detecting about 90% of the injected vulnerabilities.

III. PROPOSED STUDY: TEST-MENDER

Inspired by CodeMender’s agent-based repair framework, which uses structured tool feedback for automated code correction, we propose to adapt this paradigm to a novel task: generating executable security test cases. While CodeMender focuses on remediation through patch synthesis, our system TestMender targets detection by converting vulnerability evidence into reproducible tests.

A. Objective

The primary objective of this study is to investigate whether Large Language Models (LLMs), when guided by structured vulnerability evidence, can autonomously generate effective security test cases. Specifically, we aim to produce executable C test harnesses capable of exposing known weaknesses by forcing failure on vulnerable implementations and passing on corrected versions.

B. Dataset

We will evaluate TestMender using the Juliet Test Suite, a widely recognized benchmark containing thousands of paired

bad() (vulnerable) and good() (patched) functions across diverse CWE categories (e.g., buffer overflows, integer errors). These pairs offer an ideal ground truth for differential test validation.

C. Novelty

Unlike prior work that relies on prompting for test generation, our approach adopts an agent-based workflow inspired by CodeMender. Rather than producing code patches, TestMender repurposes this multi-stage paradigm tool integration, validation, and refinement loops for security-focused test synthesis. This architectural shift transforms LLMs from passive generators into iterative agents capable of reasoning over evidence, validating outputs, and improving tests through feedback.

D. Measurement

Test quality will be measured using:

- 1) Differential Correctness: test fails on vulnerable code and passes on patched code.
- 2) CWE Coverage: number of vulnerability classes for which tests are successfully generated.
- 3) Execution Validity: compilation rate and runtime stability of generated harnesses.

E. Procedure

The system starts with an **Evidence Package** as input, which is processed by the TestMender LLM Agent (evidence Proof of Concept/Test). This agent utilizes its Agent Environment and Tools For Agent (such as static checks, fuzzing, etc.) to generate 'harnesses'. These harnesses are passed to the **'Validator'** which uses validation Tools to produce **'Validated Edits'**. The process concludes with the **Final Test Suite**, which can provide feedback to agent for iterative improvement. Fig. 1 is displaying the proposed flow chart of the project, TestMender.

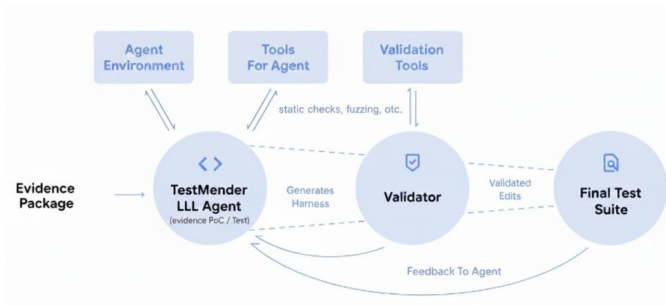


Fig. 1. Proposed workflow of TestMender

REFERENCES

- [1] Xu, D., Tu, M., Sanford, M., Thomas, L., Woodraska, D., Xu, W. (2012). "Automated security test generation with formal threat models." IEEE transactions on dependable and secure computing, 9(4), 526-540.
- [2] Test, J. "The Juliet 1.1 C/C++ and Java Test Suite"
- [3] R. A. Popa and F. Flynn, "Introducing CodeMender: An AI Agent for Code Security," Google DeepMind Blog, Oct. 6, 2025. [Online]. Available: <https://deepmind.google/discover/blog/introducing-codemender-an-ai-agent-for-code-security/>